CG303 FAULT LAB

CG303 Fault Lab - Simulator Specification

Document version: 1.1
Simulator baseline: multi-fault combination release
Last updated: 15 July 2026
Status: maintained project specification

USER MANUAL | SPECIFICATION

1. Purpose

CG303 Fault Lab is an independent browser-based training simulator for learners preparing for City & Guilds 2365-03 Unit 303, Electrical installations: fault diagnosis and rectification. It provides original simulated exercises in safe isolation, instrument selection, evidence-led testing, diagnosis, recommended correction and reporting.

It is not an official City & Guilds product, does not reproduce live assessment material, does not certify competence and does not replace supervised practical training with real installations and instruments.

2. Supported platforms

- Current desktop versions of Edge, Chrome, Firefox and Safari.
- Current iPhone and iPad Safari.
- Current Android Chrome on phones and tablets.
- Portrait and landscape layouts down to 320 CSS pixels wide.
- Keyboard, mouse, touch and switch-compatible sequential controls.
- Static web hosting, including GitHub Pages.
- Production address: `https://ckleung17.github.io/cg303-simulator/`.
- Offline use after the service worker has cached the application.

JavaScript ES modules and service-worker features require the simulator to be served over HTTP/HTTPS rather than opened directly as a local file.

3. Application modes

Guided mode

Displays explanatory feedback after readings and safety actions. Intended for learning unfamiliar diagnostic processes.

Practice mode

Records readings with reduced guidance. Intended for repeated independent work.

Exam-style mode

Minimises immediate instructional feedback and delays the overall evaluation until the report. It remains original practice content, not an official mock.

4. Circuit catalogue

The scenario generator currently covers:

- Radial socket circuits.
- Ring-final circuits.
- One- and two-way lighting circuits.
- Dedicated cooker and shower radial circuits.
- Contactor, start/stop and auxiliary holding-control circuits.
- Three-phase motor and distribution circuits.

The catalogue contains 13 fault definitions, including conductor discontinuities, reversed polarity, low insulation resistance, a high-resistance termination, an open contactor coil, an open holding contact and phase loss.

Each generated scenario activates between one and three compatible faults from one circuit family. The active-fault order is shuffled by the scenario seed, so fault count, symptoms, evidence sequence and required diagnosis are not fixed.

5. Scenario data contract

Each fault definition provides:

- Unique identifier and circuit family.
- Circuit title, supply, protection, conductors and previous result.
- Customer complaint and graphical device/wire labels.
- Available labelled test terminals.
- One or more discriminating key tests.
- Measurement behaviour for the hidden fault.
- Accepted diagnosis and recommended corrective action.

A hexadecimal scenario seed drives a deterministic pseudo-random generator. It selects a circuit family, shuffles that family's compatible fault definitions and activates one, two or three faults up to the available family maximum. Repeating the same seed in the same ruleset recreates the same ordered combination.

The composite scenario merges customer symptoms, key tests, measurement overrides, accepted diagnoses and corrective actions. When a selected test is a key test for an active fault, that fault supplies the reading; otherwise the healthy baseline rule applies. Faults from different circuit families are never combined.

The optional `-seed=` URL parameter accepts one to eight hexadecimal characters and provides a reproducible scenario deep link.

6. Diagnostic workflow

The interface presents this sequence as a responsive five-block SVG diagram with a concise text alternative. It remains legible on desktop and stacks with its introductory text at tablet and phone widths.

1. Review the customer complaint, circuit information and previous result.
2. Record initial information-gathering actions.
3. Complete the six-stage safe-isolation sequence in order.
4. Select a suitable instrument and function.
5. Select two labelled terminals and take a simulated reading.

6. Record sufficient evidence to identify the fault.

7. Select every fault supported by the evidence and a corrective-action category.

8. Explain the reasoning and review the curriculum-mapped report.

Unsafe or out-of-sequence isolation actions are recorded and reduce the safety score. Testing is blocked until safe isolation is complete.

7. Measurement model

The simulator uses deterministic teaching rules rather than transient circuit analysis. Each scenario overrides its discriminating measurement; compatible non-faulted paths use baseline rules:

- Same-conductor continuity normally returns a low resistance.
- Separate conductors normally return open circuit for continuity.
- Separate healthy conductors normally return high insulation resistance.
- Tests on an isolated circuit return zero measured voltage.

Displayed values are educational simulations. They must not be interpreted as instructions or expected values for an unverified real installation.

8. Assessment model

The report allocates 100 marks:

- Health and safety: 25.
- Information gathering and communication: 15.
- Test selection and evidence: 25.
- Complete diagnosis of the active fault set: 20.
- Corrective-action category: 10.
- Written reasoning: 5.

Results are also mapped to the six published Unit 303 learning-outcome themes.

A simulator score is formative feedback and is not a qualification result.

9. Accessibility and responsive requirements

- Semantic headings, forms, fieldsets, labels, status regions and tables.
- Visible keyboard focus and a skip link.
- Touch targets at least 44 by 44 CSS pixels.
- No task dependent exclusively on hover, colour or drag-and-drop.
- Terminal buttons provide keyboard/touch alternatives to graphical probe leads.
- Mobile safe-area, text enlargement and orientation support.
- Switchable/single-column layout at phone widths.
- Centred action-button labels and separately stacked manual-download links at phone widths.

10. Technical structure

- `index.html`: semantic application shell.
- `css/`: tokens, foundations, layouts and simulator components.

- `js/app.js`: application orchestration and learner workflow.
- `data/scenarios/`: circuit, terminal, fault and measurement content.
- `assets/`: original SVG and raster resources.
- `service-worker.js` and `manifest.webmanifest`: installable/offline support.
- `tests/verify_project.py`: dependency-free structural regression checks.
- `docs/`: maintained requirements, design records and manuals.

The application has no runtime backend, account system or external analytics.

Attempt summaries are stored locally in the browser when local storage is

available.

11. Verification requirements

Every release must:

1. Run `python tests/verify_project.py` successfully.

2. Pass `git diff --check`.

3. Load through a local HTTP server without JavaScript module errors.

4. Be checked at representative phone, tablet and desktop viewports.

5. Preserve keyboard and touch access to the full diagnostic workflow.

6. Update this specification and the operation manual when behaviour changes.

12. Known limitations and planned development

- The electrical engine is a deterministic rule model, not a general-purpose

network solver.

- Compatibility is currently enforced at circuit-family level; more detailed

masking and interaction constraints remain an area for expansion.

- Simulated correction is selected and reported; components are not graphically

dismantled or rewired.

- Formal technical review metadata is still required for released scenario data.
- Automated end-to-end browser coverage should be expanded.

13. Document maintenance

This specification is a controlled project document. Any amendment affecting

features, circuit coverage, scoring, measurements, safety workflow, supported

devices, accessibility, storage or architecture must update this file in the

same commit. Update the version, date and simulator baseline when preparing a

release.